PostgreSQL HANDBOOK

★ What is PostgreSQL?

PostgreSQL is a powerful, open source object-relational database system. It uses and extends the SQL language combined with many features that safely store and scale the most complicated data workloads.

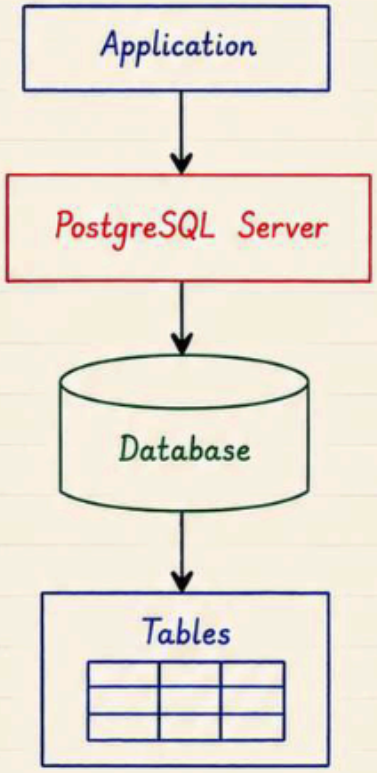
★ Why PostgreSQL?

- Open Source and Free
- ACID Compliant
- High Performance & Scalability
- Extensible (Functions, Types, Operators)
- Strong Security
- Rich Feature Set

★ Features

- Supports Advanced SQL
- JSON / JSONB Support
- Indexes (B-Tree, Hash, GIN, GiST)
- Views, Functions, Triggers
- Full Text Search
- Replication & Backup

Architecture



Where is it used?

- Web Applications
- Analytics & Reporting
- GIS Applications
- Financial Systems
- AI / ML Applications

Try this in psql

```
postgres=# SELECT version();
                          version
-----------------------------------------------------------
PostgreSQL 16.2 (Debian 16.2-1.pgdg120+1)
(1 row)
```

Note :
PostgreSQL runs on Linux, Windows, macOS and all major platforms.

★ Database Operations

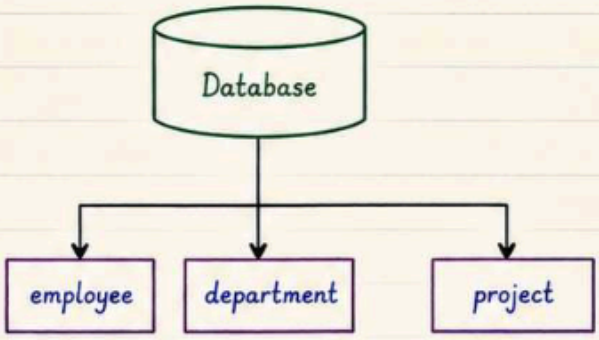
- **Create Database** → Creates a new database.

```
CREATE DATABASE company;
```

- **Drop Database** → Deletes an existing database.

```
DROP DATABASE company;
```

Database Structure



★ A database contains one or more tables.

★ Table Operations

- **Create Table** → Creates a new table in the database.

```
CREATE TABLE employee (
  id      SERIAL PRIMARY KEY,
  name    VARCHAR(100) NOT NULL,
  email   VARCHAR(100) UNIQUE,
  salary  NUMERIC(10,2),
  joined_on  DATE DEFAULT CURRENT_DATE
);
```

Table Structure (employee)

Column Name	Data Type	Constraint
id	SERIAL	PRIMARY KEY
name	VARCHAR(100)	NOT NULL
email	VARCHAR(100)	UNIQUE
salary	NUMERIC(10,2)	—
joined_on	DATE	DEFAULT

- **Alter Table** → Modifies the structure of an existing table.

```
ALTER TABLE employee ADD COLUMN age INT;
ALTER TABLE employee RENAME COLUMN name TO full_name;
ALTER TABLE employee DROP COLUMN age;
```

- **Drop Table** → Deletes an existing table.

```
DROP TABLE employee;
```

- **Rename Table** → Changes the name of a table.

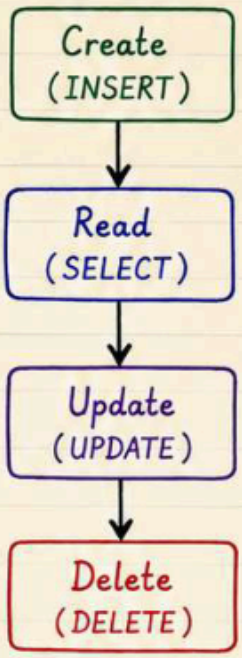
```
ALTER TABLE employee
RENAME TO emp;
```

💡 Pro Tip: Use meaningful names for databases and tables. It makes your queries easy to understand.

★ CRUD Operations

CRUD → Create, Read, Update, Delete

These are the basic operations we perform on any table.



- **INSERT** → Add new data into table.

```
INSERT INTO employee (name, email, salary, joined_on)
VALUES ('Aman', 'aman@example.com', 75000,
        '2024-05-20');
-- Adds one new row into employee table
```

- **SELECT** → Retrieve data from table.

```
SELECT id, name, email, salary
FROM employee
WHERE salary > 50000
ORDER BY salary DESC;
-- Reads data based on condition
```

Common Clauses

- WHERE → filter rows
- ORDER BY → sort result
- LIMIT → limit rows
- DISTINCT → unique rows

- **UPDATE** → Modify existing data.

```
UPDATE employee
SET salary = 80000
WHERE id = 1;
-- Updates salary for employee with id = 1
```

💡 Tip:
Always use WHERE clause while UPDATE to avoid updating all rows.

- **DELETE** → Remove data from table.

```
DELETE FROM employee
WHERE id = 1;
-- Deletes row where id = 1
```

💡 Tip:
Use DELETE carefully. Once deleted, data cannot be recovered.

☑ Key Points

- INSERT adds new rows.
- SELECT reads data.
- UPDATE modifies existing rows.
- DELETE removes rows.
- Always use WHERE clause with UPDATE and DELETE.
- Commit changes using transactions for data safety.

★ Joins

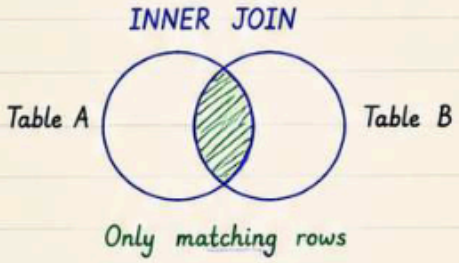
Joins combine rows from two or more tables based on a related column between them.

Why Joins?

- To fetch meaningful data from multiple tables.
- Core of relational databases.

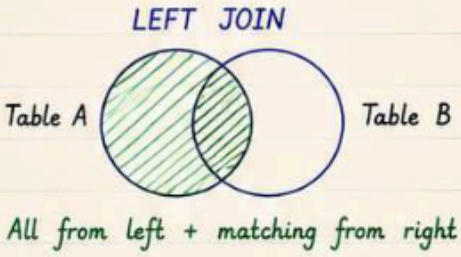
① INNER JOIN → Returns only matching rows.

```
SELECT  e.name, d.dept_name
FROM    employee e
INNER JOIN department d
ON      e.dept_id = d.dept_id;
```



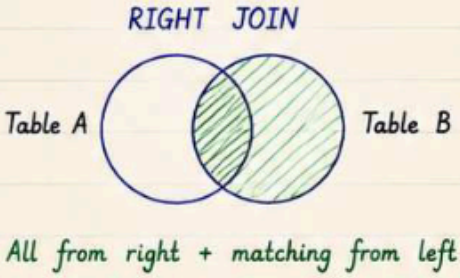
② LEFT JOIN → Returns all rows from left table + matching from right.

```
SELECT  e.name, d.dept_name
FROM    employee e
LEFT JOIN department d
ON      e.dept_id = d.dept_id;
```



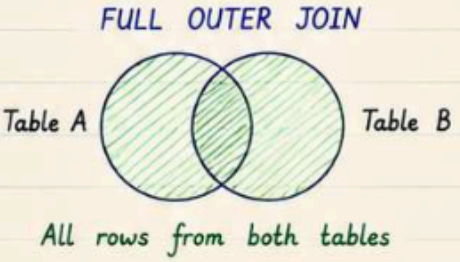
③ RIGHT JOIN → Returns all rows from right table + matching from left.

```
SELECT  e.name, d.dept_name
FROM    employee e
RIGHT JOIN department d
ON      e.dept_id = d.dept_id;
```



④ FULL OUTER JOIN → Returns all rows from both tables.

```
SELECT  e.name, d.dept_name
FROM    employee e
FULL OUTER JOIN department d
ON      e.dept_id = d.dept_id;
```



Example Tables

employee

id	name	dept_id
1	Aman	10
2	Priya	20
3	Rahul	NULL
4	Neha	30

department

dept_id	dept_name
10	HR
20	IT
30	Finance
40	Marketing

Pro Tip:

- Use table aliases (e, d) to write short & clean queries.
- Always use meaningful joins based on relationships.
- Check NULLs in LEFT / RIGHT joins.

★ Indexes

Indexes improve the speed of data retrieval operations on a table.

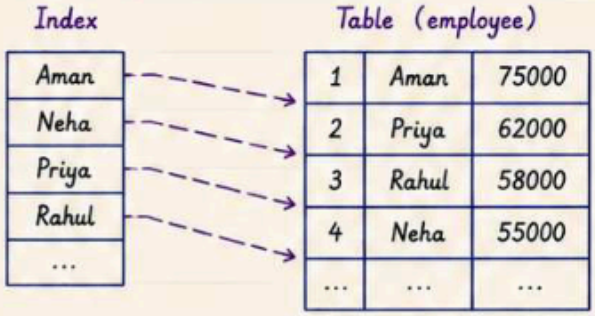
Why Indexes?

- Reduce query execution time.
- Minimize disk I/O.
- Help in fast searching, sorting and filtering.

- **Create Index** → Creates an index on a column.

```
CREATE INDEX idx_employee_name
ON employee (name);
-- Creates an index named idx_employee_name
on column name of employee table
```

How Index Works?



Index points to actual data location.

- **Unique Index** → Ensures all values in indexed column are unique.

```
CREATE UNIQUE INDEX idx_employee_email
ON employee (email);
-- Prevents duplicate values in email column
```

Types of Indexes

- B-Tree Index → Default index type.
- Hash Index → For equality searches.
- GiST Index → For complex data types (geometry, text search).
- GIN Index → For full text search and array data.
- BRIN Index → For large tables with ordered data.

- **Drop Index** → Removes an existing index.

```
DROP INDEX idx_employee_name;
-- Drops the index from the table
```

- **List Indexes on a Table** → Shows all indexes for a table.

```
SELECT indexname, indexdef
FROM pg_indexes
WHERE tablename = 'employee';
-- Lists all indexes on employee table
```

Best Practices

- ☑ Index columns used in WHERE, JOIN, ORDER BY, GROUP BY.
- ☑ Do not over-index. It slows down INSERT, UPDATE, DELETE.
- ☑ Monitor unused indexes and remove them.
- ☑ Use appropriate index type for your use case.

- **Check Index Usage** → Understand if index is being used.

```
EXPLAIN ANALYZE
SELECT * FROM employee WHERE name = 'Aman';
-- Shows query plan and index usage
```

Pro Tip:

Use EXPLAIN ANALYZE to verify if your queries are using indexes effectively.

PostgreSQL Handbook

★ Functions

Functions perform calculations, manipulate data and return a single value.

Types of Functions

- Built-in Functions → Provided by PostgreSQL
- Aggregate Functions → Work on a set of rows
- User Defined Functions (UDF) → Custom functions

- **Built-in Functions** → Commonly used built-in functions.

Category	Function	Example
String	UPPER()	SELECT UPPER('aman'); -- AMAN
String	LOWER()	SELECT LOWER('AMAN'); -- aman
Date/Time	NOW()	SELECT NOW(); -- current timestamp
Math	ABS()	SELECT ABS(-25); -- 25
Math	ROUND()	SELECT ROUND(1234.567,2); -- 1234.57
Null Handling	COALESCE()	SELECT COALESCE(NULL,'N/A'); -- N/A

- **Aggregate Functions** → Work on multiple rows

```
COUNT()   → SELECT COUNT(*) FROM employee;
SUM()     → SELECT SUM(salary) FROM employee;
AVG()     → SELECT AVG(salary) FROM employee;
MIN()     → SELECT MIN(salary) FROM employee;
MAX()     → SELECT MAX(salary) FROM employee;
GROUP BY  → GROUP BY dept_id;
    -- Used with GROUP BY to summarize data
```

- **User Defined Functions (UDF)** → Create custom logic.

```
CREATE OR REPLACE FUNCTION get_bonus(emp_id INT)
RETURNS NUMERIC AS $$
DECLARE
    bonus NUMERIC;
BEGIN
    SELECT salary * 0.1 INTO bonus
    FROM employee
    WHERE id = emp_id;
    RETURN bonus;
END;
$$ LANGUAGE plpgsql;
```

> **Tip:** UDFs help reuse complex business logic.

★ Views

A view is a virtual table based on the result of a SELECT query.

- **Create View**

```
CREATE VIEW high_earners AS
SELECT id, name, salary
FROM employee
WHERE salary > 50000;
```

- **Use View**

```
SELECT * FROM high_earners;
```

- **Drop View**

```
DROP VIEW high_earners;
```

> **Tip:** Views simplify complex queries and enhance security.

★ Schema, Search Path & Aliases

- **Schema** Logical container for database objects.

```
CREATE SCHEMA hr;
CREATE TABLE hr.employee (id SERIAL PRIMARY KEY);
```

- **Search Path** Determines the order in which schemas are searched.

```
SET search_path TO hr, public;
```

- **Table Alias** Short name for tables in queries.

```
SELECT e.name, e.salary FROM employee e WHERE e.salary > 50000;
```

Best Practices

- ☑ Use meaningful function and view names.
- ☑ Avoid SELECT * in views and functions.
- ☑ Use SCHEMA to organize objects logically.
- ☑ Prefer built-in functions before writing UDFs.
- ☑ Keep functions small, reusable and well tested.
- ☑ Use views to restrict access and simplify queries.

Pro Tip: Leverage PostgreSQL functions and views to write clean, efficient and maintainable SQL!

PostgreSQL Handbook

★ Constraints

Constraints enforce rules on data to ensure accuracy and integrity.

Constraint	Description	Example
PRIMARY KEY	Uniquely identifies each row.	id SERIAL PRIMARY KEY
FOREIGN KEY	References a column in another table.	dept_id INT REFERENCES dept(id)
UNIQUE	Ensures all values are unique.	email VARCHAR(100) UNIQUE
NOT NULL	Column cannot have NULL.	name VARCHAR(100) NOT NULL
CHECK	Ensures values satisfy a condition.	salary NUMERIC CHECK (salary > 0)
DEFAULT	Sets default value for a column.	status VARCHAR(20) DEFAULT 'active'

Why Constraints?

- Maintain data accuracy.
- Prevent invalid data.
- Enforce business rules.
- Improve reliability of the database.

- **Add Constraints** → Using ALTER TABLE.

```
ALTER TABLE employee
  ADD CONSTRAINT uq_email UNIQUE (email);

ALTER TABLE employee
  ADD CONSTRAINT chk_salary CHECK (salary > 0);

ALTER TABLE employee
  ALTER COLUMN name SET NOT NULL;

ALTER TABLE employee
  ADD CONSTRAINT fk_dept FOREIGN KEY (dept_id)
    REFERENCES department(id);
```

- **Drop Constraints** → Remove existing constraints.

```
ALTER TABLE employee DROP CONSTRAINT uq_email;

ALTER TABLE employee DROP CONSTRAINT chk_salary;

ALTER TABLE employee DROP CONSTRAINT fk_dept;
```

- **Constraint on Table Creation** → Define while creating.

```
CREATE TABLE department (
  id SERIAL PRIMARY KEY,
  name VARCHAR(100) NOT NULL UNIQUE
);

CREATE TABLE employee (
  id SERIAL PRIMARY KEY,
  name VARCHAR(100) NOT NULL,
  email VARCHAR(100) UNIQUE,
  salary NUMERIC CHECK (salary > 0),
  dept_id INT REFERENCES department(id)
);
```

★ Sequences

Sequences generate unique numeric values.

```
CREATE SEQUENCE emp_seq START 1 INCREMENT 1;
SELECT nextval('emp_seq');   -- Next value
SELECT currval('emp_seq');   -- Current value
ALTER SEQUENCE emp_seq RESTART WITH 100;
```

★ Transactions (ACID)

Transactions ensure data reliability.

- **A – Atomicity** : All or nothing.
- **C – Consistency** : Database remains consistent.
- **I – Isolation** : Transactions are isolated.
- **D – Durability** : Changes are permanent.

Example

```
BEGIN;
UPDATE employee SET salary = salary + 1000
  WHERE id = 1;
UPDATE employee SET salary = salary + 1000
  WHERE id = 2;
COMMIT;  -- Save changes
```

```
ROLLBACK;  -- Undo changes
```

Best Practices

- ☑ Always define PRIMARY KEY.
- ☑ Use NOT NULL and UNIQUE whenever possible.
- ☑ Use FOREIGN KEY to maintain relationships.
- ☑ Use CHECK constraints for business rules.
- ☑ Keep sequences and constraints meaningful and consistent.
- ☑ Test constraints before deploying.

Pro Tip: Good constraints today = Clean and reliable data forever!

PostgreSQL Handbook

★ Triggers

Triggers automatically execute a function in response to an event on a table.

• Trigger Types

Type	Description
BEFORE Trigger	Executes before the event (INSERT, UPDATE, DELETE).
AFTER Trigger	Executes after the event.
INSTEAD OF Trigger	Used with views instead of the event.
ROW Level Trigger	Fires once for each row affected.
STATEMENT Level Trigger	Fires once for the whole statement.

• Create Function (Example)

```
CREATE OR REPLACE FUNCTION set_updated_at()
RETURNS TRIGGER AS $$
BEGIN
    NEW.updated_at = NOW();
    RETURN NEW;
END;
$$ LANGUAGE plpgsql;
```

• Create Trigger

```
CREATE TRIGGER set_timestamp
BEFORE UPDATE ON employee
FOR EACH ROW
EXECUTE FUNCTION set_updated_at();
```

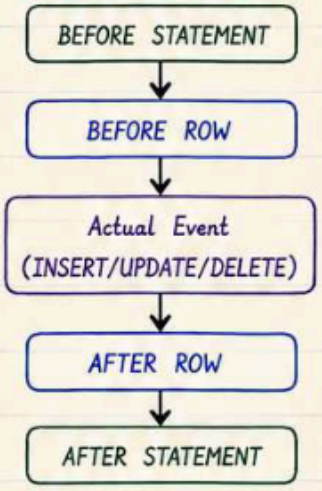
Why Triggers?

- Enforce business rules.
- Maintain audit logs.
- Keep data consistent.
- Automate repetitive tasks.

• Example Use Case

Automatically update "updated_at" column whenever a row is updated.

Trigger Execution Order



Tip:

For most use cases, ROW level triggers are preferred.

★ Stored Procedures

Stored procedures are precompiled SQL code that can accept parameters.

• Create Procedure

```
CREATE OR REPLACE PROCEDURE give_bonus(emp_id INT,
                                       bonus NUMERIC)
LANGUAGE plpgsql
AS $$
BEGIN
    UPDATE employee
    SET salary = salary + bonus
    WHERE id = emp_id;
END;
$$;
```

• Call Procedure

```
CALL give_bonus(1, 5000);
```

• Advantages

- Reusability.
- Better security.
- Encapsulation of business logic.
- Easy maintenance.

Best Practices

- ☑ Keep triggers lightweight.
- ☑ Avoid complex logic in triggers.
- ☑ Name triggers and functions meaningfully.
- ☑ Use stored procedures for repetitive tasks.
- ☑ Document your triggers and procedures.

Pro Tip: Triggers and stored procedures are powerful. Use them wisely to keep your database Fast, Consistent and Smart!